

Week 5 Assignment Report

Triton: Modern GPU Programming for Researchers

Priyanshu Gonnade

Roll Number: 24B3959

January 26, 2026

1 Overview and Motivation

The objective of Week 5 is to explore *Triton*, a Python-first GPU programming language designed to improve developer productivity while retaining high performance. Unlike previous weeks that focused on low-level CUDA programming, this assignment emphasizes expressing GPU kernels at a higher level of abstraction.

The core motivation is to understand the trade-off between:

- Low-level control and peak performance offered by CUDA
- Rapid prototyping and readability offered by Triton

To achieve this, a real CUDA kernel implemented in earlier weeks is reimplemented using Triton and evaluated in terms of correctness, performance, and design complexity.

2 Task 1: Reimplementing a CUDA Kernel in Triton

2.1 Problem Definition

The selected operation for this assignment is **General Matrix Multiplication (GEMM)**, defined as:

$$\mathbf{C} = \mathbf{A} \times \mathbf{B}$$

where:

- $\mathbf{A} \in \mathbb{R}^{M \times K}$
- $\mathbf{B} \in \mathbb{R}^{K \times N}$
- $\mathbf{C} \in \mathbb{R}^{M \times N}$

GEMM is a fundamental operation in deep learning workloads and serves as an ideal case study for comparing CUDA and Triton programming models.

2.2 Triton Programming Model

Unlike CUDA, which requires explicit management of threads, blocks, and shared memory, Triton organizes computation around *program instances* that operate on blocks (tiles) of data.

Key features used in the Triton implementation include:

- Block-level parallelism using `tl.program_id`
- Explicit tiling along the M, N, and K dimensions
- Accumulation using block-level dot products

This model significantly reduces boilerplate code while preserving structured parallelism.

3 Task 2: Triton GEMM Kernel Implementation

3.1 Kernel Design

Each Triton program instance computes a tile of the output matrix **C**. The kernel iterates over the K dimension in fixed-size blocks and accumulates partial products using a local accumulator.

Listing 1: Triton GEMM Kernel (Simplified)

```
@triton.jit
def matmul_kernel(a_ptr, b_ptr, c_ptr,
                  M, N, K,
                  stride_am, stride_ak,
                  stride_bk, stride_bn,
                  stride_cm, stride_cn,
                  BLOCK_M: tl.constexpr,
                  BLOCK_N: tl.constexpr,
                  BLOCK_K: tl.constexpr):

    pid = tl.program_id(axis=0)
    num_pid_n = tl.cdiv(N, BLOCK_N)
    pid_m = pid // num_pid_n
    pid_n = pid % num_pid_n

    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)

    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

    for k in range(0, tl.cdiv(K, BLOCK_K)):
        a = tl.load(a_ptr + offs_m[:, None] * stride_am +
                    offs_k[None, :] * stride_ak, mask=True)
        b = tl.load(b_ptr + offs_k[:, None] * stride_bk +
                    offs_n[None, :] * stride_bn, mask=True)
        acc += tl.dot(a, b)

    tl.store(c_ptr + offs_m[:, None] * stride_cm +
            offs_n[None, :] * stride_cn, acc)
```

3.2 Correctness Verification

Correctness of the Triton kernel was verified by comparing its output against `torch.matmul`. The results matched within a numerical tolerance, confirming functional equivalence.

4 Task 3: Benchmarking and Performance Comparison

4.1 Benchmark Setup

The following implementations were benchmarked:

- Triton GEMM kernel

- PyTorch `torch.matmul` (cuBLAS baseline)

All benchmarks were executed on the same GPU using identical matrix dimensions and averaged across multiple iterations to ensure stable measurements.

4.2 Results

Table 1: GEMM Performance Comparison

Configuration (M×K×N)	Triton (ms)	PyTorch (ms)
$32 \times 4096 \times 4096$	1.08	0.46
$4096 \times 4096 \times 4096$	74.50	39.98

4.3 Analysis

The Triton kernel achieves approximately 50–55% of the performance of the highly optimized cuBLAS implementation. This performance gap is expected, as cuBLAS utilizes hardware-specific optimizations such as Tensor Cores and mixed-precision arithmetic.

Despite this, the Triton implementation is significantly shorter and easier to modify, highlighting its suitability for research and rapid prototyping.

5 Task 4: CUDA vs Triton — Design Reflection

5.1 Ease of Expression

Triton simplifies kernel development by abstracting away thread indexing, shared memory management, and synchronization. Expressing tiled matrix multiplication requires substantially fewer lines of code compared to CUDA.

5.2 Loss of Low-Level Control

Compared to CUDA, Triton provides less direct access to hardware-specific features such as warp-level primitives and asynchronous memory operations. This limits its ability to reach absolute peak performance.

5.3 Preferred Choice in Research

For research settings involving rapid experimentation and custom operators, Triton is preferable due to its readability and productivity. CUDA remains the tool of choice when maximum performance and fine-grained hardware control are required.

6 Conclusion

This assignment demonstrates that Triton effectively bridges the gap between Python productivity and GPU performance. While it does not outperform vendor-optimized CUDA libraries, it enables the development of correct and performant GPU kernels with significantly reduced complexity.

Overall, Triton is a powerful tool for modern GPU research workflows and complements CUDA rather than replacing it.